

Universidad de Valladolid

Grado en Informática

Evaluación de Sistemas Informáticos

Práctica 1: Monitorización del sistema y Grafana

Iván Moro Cienfuegos

ivan.moro22@estudiantes.uva.es

Pablo March Ortega

pablo.march23@estudiantes.uva.es

Nicolás Reyes Gutiérrez

nicolas.reyes@estudiantes.uva.es

Marzo 2026



Universidad de Valladolid

Índice

1. Introducción	3
2. Instalación y acceso al servidor	3
2.1. Acceso al servidor	3
2.2. Instalación de Dependencias y Base de Datos	3
3. Programa de recolección de métricas	4
3.1. Descripción e Implementación del Servicio	4
3.2. Métricas Avanzadas Recogidas	4
4. Visualización en Grafana	5
4.1. Paneles de Hardware y Sistema Base	5
4.1.1. Distribucion estadística de carga de CPU	5
4.1.2. Porcentaje de uso de cada núcleo de CPU	6
4.1.3. Carga del sistema	6
4.1.4. Operaciones de disco	7
4.1.5. Espacio en disco	7
4.1.6. Uso de RAM y espacio Swap	8
4.1.7. Número de procesos	8
4.1.8. Estadísticas de uso de MariaDB	9
4.2. Paneles de Red, Seguridad y Servicios	9
4.2.1. Tráfico de Red	9
4.2.2. Desglose tiempo respuesta HTTPS	10
4.2.3. Estadísticas de sesión del servidor	10
4.2.4. Puertos TCP abiertos	11
4.2.5. Topología de red local (ARP)	11
4.2.6. Árbol de procesos (usando CPU)	12
4.3. Visualizaciones Avanzadas (ECharts)	12
5. Pruebas de Estrés y Validación	13
5.1. Scripts de Prueba Desarrollados	13
5.2. Resultados Observados	14
6. Conclusión	14

1. Introducción

Todos los sistemas informáticos modernos deben garantizar dos requisitos no funcionales esenciales: un funcionamiento correcto y un rendimiento adecuado. Para poder asegurar que un sistema cumple estos requisitos debemos ser capaces de obtener datos del funcionamiento del mismo. En esta primera práctica, el objetivo será elaborar un sistema avanzado de observabilidad que recoja métricas del sistema (hardware, red, bases de datos y servicios) y visualizar estas métricas utilizando Grafana.

2. Instalación y acceso al servidor

2.1. Acceso al servidor

Los datos de acceso a nuestra máquina virtual asignada son los siguientes:

```
Host: virtual.lab.inf.uva.es
Usuario: usuario
Password: 2Moronipa.
Puerto SSH: 31081
Puerto Grafana (3000): 31082
Credenciales grafana: Usuario: admin Password: 2Moronipa.
Credenciales mysql: Usuario: elliot Password: 2Moronipa.
```

Datos de Conexión

La conexión al sistema se realiza mediante el protocolo SSH apuntando al puerto 31081 a través del host `virtual.lab.inf.uva.es`.

2.2. Instalación de Dependencias y Base de Datos

Para cumplir con los requisitos del proyecto, se ha instalado **MariaDB** por su eficiencia y compatibilidad nativa con Grafana. Adicionalmente, se instaló **Python 3** junto con las librerías `psutil` y `mysql-connector-python`.

Se ha diseñado un esquema de base de datos relacional avanzado para soportar métricas dinámicas:

```
CREATE DATABASE ESI_Practical;
USE ESI_Practical;

-- Tabla principal de métricas históricas
CREATE TABLE registro_metricas (
  id INT AUTO_INCREMENT PRIMARY KEY,
  timestamp DATETIME NOT NULL,
  cpu_percent FLOAT, cpu_core_1 FLOAT, cpu_core_2 FLOAT,
  ram_usada_mb FLOAT, ram_disponible_mb FLOAT, ram_percent FLOAT, swap_usada_mb FLOAT,
  disco_count_read BIGINT, disco_count_write BIGINT, disco_usado_porcentaje FLOAT,
  disco_usado_bytes BIGINT,
  red_bytes_enviados BIGINT, red_bytes_recibidos BIGINT,
  num_procesos INT, load_1m FLOAT,
  sesiones_ssh INT, uptime_horas FLOAT,
  dns_ms FLOAT, tcp_ms FLOAT, tls_ms FLOAT, server_ms FLOAT, transfer_ms FLOAT,
  mysql_qps FLOAT, mysql_threads INT, mysql_slow_queries INT, mysql_aborted_connects INT,
  mysql_bytes_rx_ps FLOAT, mysql_bytes_tx_ps FLOAT
);
```

```
-- Tablas dinámicas (se truncan en cada iteración)
CREATE TABLE arbol_procesos (pid INT, ppid INT, nombre VARCHAR(100), cpu_uso FLOAT,
    timestamp DATETIME);
CREATE TABLE vecinos_red (ip VARCHAR(15), mac VARCHAR(17), timestamp DATETIME);
CREATE TABLE puertos_activos (puerto INT, estado VARCHAR(20), cantidad INT, timestamp
    DATETIME);
```

Estructura de la Base de Datos

3. Programa de recolección de métricas

3.1. Descripción e Implementación del Servicio

Se ha desarrollado un script en Python (ver Anexo A) que actúa como un *agente recolector*. Para garantizar la alta disponibilidad del agente, su ejecución continua en segundo plano y su inicio automático tras cualquier reinicio del servidor, se ha descartado el uso de procesos efímeros en terminal y se ha integrado nativamente en el sistema operativo como un demonio de `systemd`.

Para ello, se creó el archivo de configuración del servicio `monitor.service` en el directorio `/etc/systemd/system/`:

```
[Unit]
Description=Monitor de sistema
After=network.target mariadb.service

[Service]
Type=simple
User=usuario
WorkingDirectory=/home/usuario/Practical1
ExecStart=/usr/bin/python3 /home/usuario/Practical1/practical1.py
Restart=always
RestartSec=10

[Install]
WantedBy=multi-user.target
```

Configuración del demonio (monitor.service)

Una vez definido el servicio, se habilitó su auto-arranque y se inició mediante los comandos estándar de administración:

```
sudo systemctl daemon-reload
sudo systemctl enable monitor.service
sudo systemctl start monitor.service
```

Activación del servicio

Este enfoque garantiza que el sistema operativo supervise el proceso constantemente, reiniciándolo automáticamente (`Restart=always`) en caso de fallo o cierre inesperado, asegurando así que la recolección de métricas hacia la base de datos sea ininterrumpida.

3.2. Métricas Avanzadas Recogidas

Más allá de las métricas requeridas (CPU, RAM, Disco y Red), se han implementado analizadores avanzados:

- **Análisis HTTPS (Latencia):** Uso de subprocesos con `curl` para desglosar el tiempo de respuesta (DNS, TCP, TLS, Servidor, Transferencia).
- **Salud de Servicios y Seguridad:** Monitorización de conexiones SSH establecidas reales (vía sockets) y Uptime del sistema.
- **Tráfico TCP local:** Escáner de puertos locales en estado LISTEN, ESTABLISHED o TIME_WAIT para detección de anomalías.
- **Rendimiento de Base de Datos (DBA):** Extracción de métricas internas de MariaDB (QPS, Threads, Tráfico de Red y Slow Queries).
- **Topología (ARP) y Árbol de Procesos:** Recolección dinámica de las tablas de enrutamiento y parentesco de procesos.

4. Visualización en Grafana

4.1. Paneles de Hardware y Sistema Base

Se han configurado paneles estándar utilizando visualizaciones *Time Series*, *Stat* y *Gauge*.

4.1.1. Distribucion estadística de carga de CPU

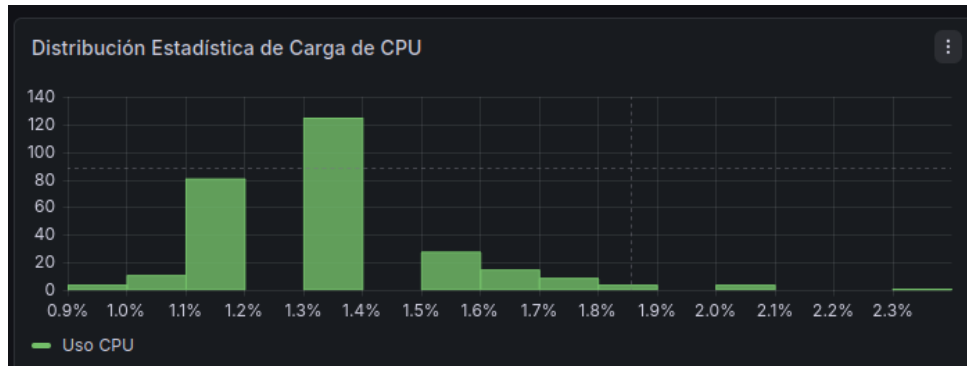


Figura 1: Distribucion estadística de carga de CPU

Grafico que muestra la distribucion estadística del porcentaje de uso global de la CPU, mostrando claramente que lo mas habitual es que la carga sea 1.1-1.2% y 1.3-1.4%.

4.1.2. Porcentaje de uso de cada nucleo de CPU



Figura 2: Porcentaje de uso de cada nucleo de CPU

Grafico que muestra el porcentaje de uso individual de cada nucleo de la CPU, permitiendonos identificar desequilibrios de carga entre nucleos y pudiendo asi balancearlo nuevamente.

4.1.3. Carga del sistema



Figura 3: Carga del sistema

Grafico que muestra la carga del sistema en el ultimo minuto. Si es superior a 1 implica que empieza a haber demasiados procesos simultaneos como para atenderlos todos , teniendo que dormir algunos. Esta estadistica tambien tiene relacion con el scheduling (SO), no puramente con el hardware.

4.1.4. Operaciones de disco



Figura 4: Operaciones de disco

Grafico que muestra la velocidad de la entrada/salida medido en operaciones/segundo.

4.1.5. Espacio en disco



Figura 5: Espacio en disco

Grafico que muestra por un lado el porcentaje de disco utilizado (%) y por otro el valor absoluto ocupado en disco (en Gibibyte).

4.1.6. Uso de RAM y espacio Swap

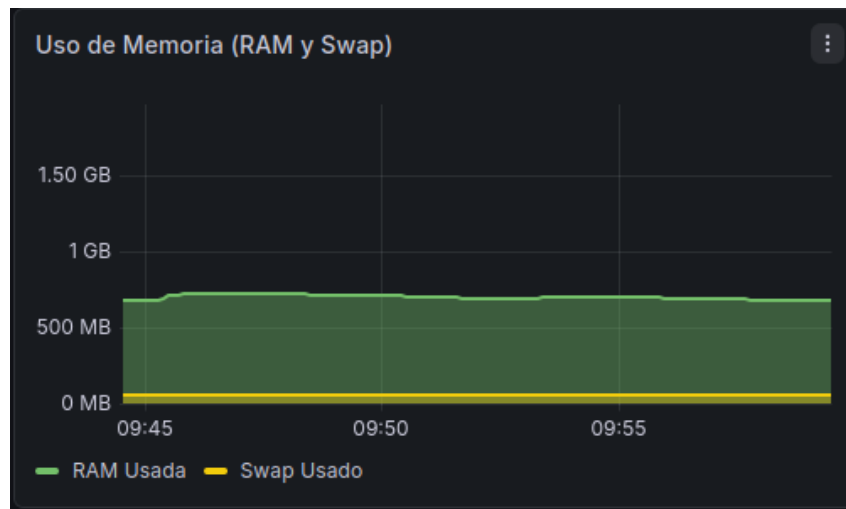


Figura 6: Uso de RAM y espacio Swap

Grafico que muestra el espacio de disco destinado a Swap y tambien el espacio ocupado de la memoria RAM.

4.1.7. Numero de procesos



Figura 7: Numero de procesos

Grafico que muestra el numero de procesos activos en el sistema.

4.1.8. Estadísticas de uso de MariaDB

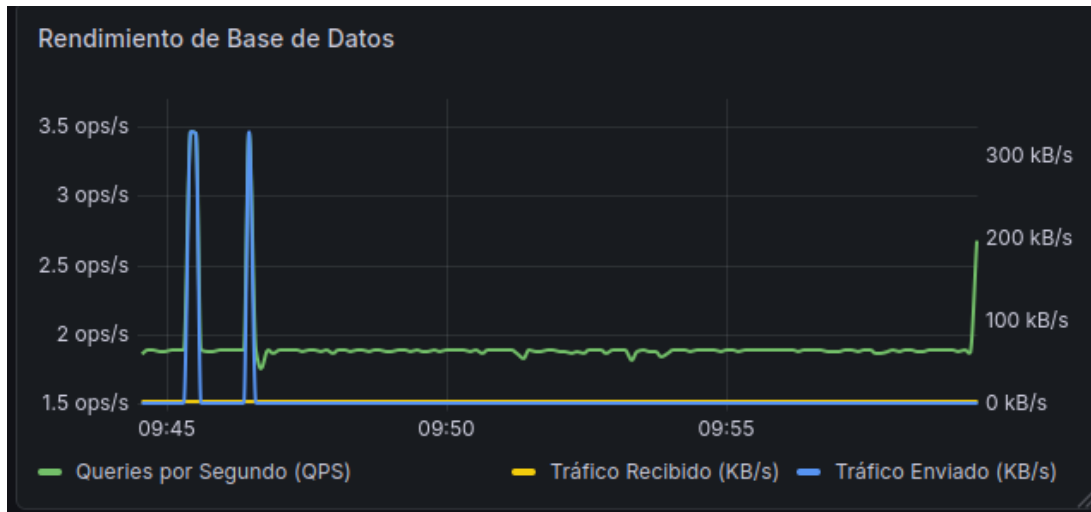


Figura 8: Estadísticas de uso de MariaDB

Gráfico que muestra el rendimiento de la base de datos, tanto la peticiones por segundo como la velocidades de entrada y salida (Kb/ s) de la base de datos en MariaDB.

4.2. Paneles de Red, Seguridad y Servicios

4.2.1. Trafico de Red

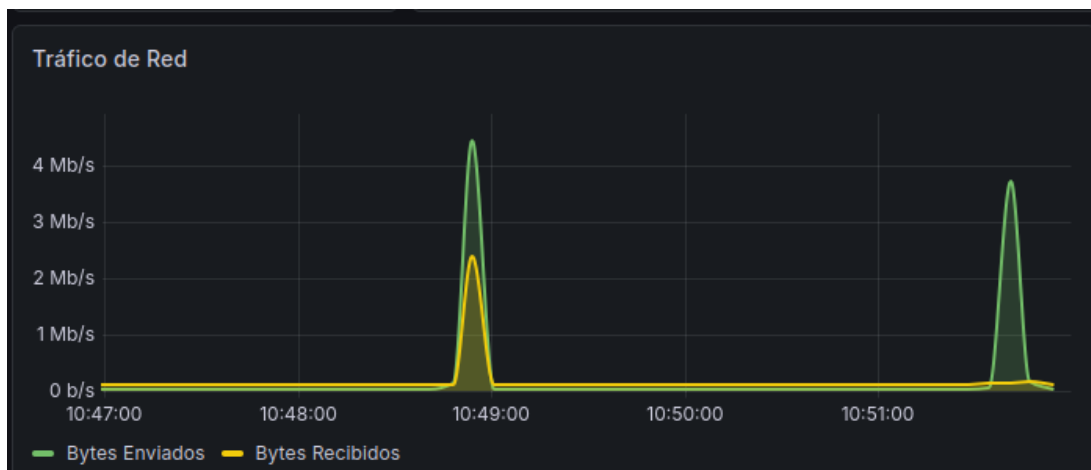


Figura 9: Trafico de Red

Gráfico que muestra el trafico de red en la maquina virtual, tanto bytes enviados como recibidos, medido en Mb/ s.

4.2.2. Desglose tiempo respuesta HTTPS

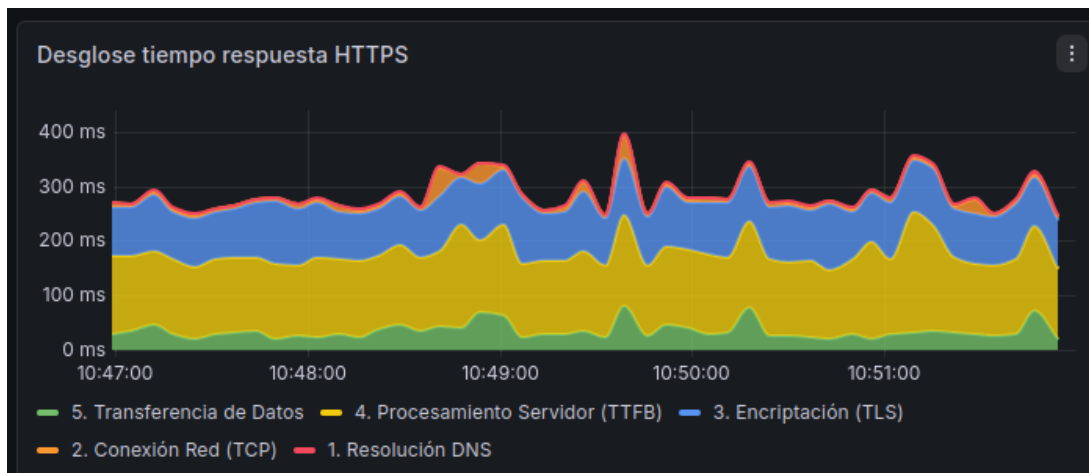


Figura 10: Desglose tiempo respuesta HTTPS

Grafico que muestra el tiempo de una respuesta HTTPS desglosado en todas sus partes, desde resolución DNS, pasando por el tiempo que tarda el three-way handshake (por usar protocolo TCP) y los tiempos de encriptación, procesamiento y transferencia real de datos.

4.2.3. Estadísticas de sesion del servidor



Figura 11: Estadísticas de sesion del servidor

Grafico que muestra por un lado el numero de sesiones SSH activas al servidor y por otro el tiempo que lleva el servidor activo (Uptime).

4.2.4. Puertos TCP abiertos

Distribución Puertos TCP			
Puerto Local	Estado de la Conexión		Conexiones Totales
22 (SSH)	ESTABLISHED		1
22 (SSH)	LISTEN		2
53 (DNS)	LISTEN		1
3000 (Grafana)	ESTABLISHED		13
3000 (Grafana)	LISTEN		1
3306 (MySQL)	ESTABLISHED		6
3306 (MySQL)	LISTEN		1

Figura 12: Puertos TCP abiertos

Grafico que muestra todos los puertos TCP del servidor con sus estados y el numero total de conexiones.

4.2.5. Topologia de red local (ARP)

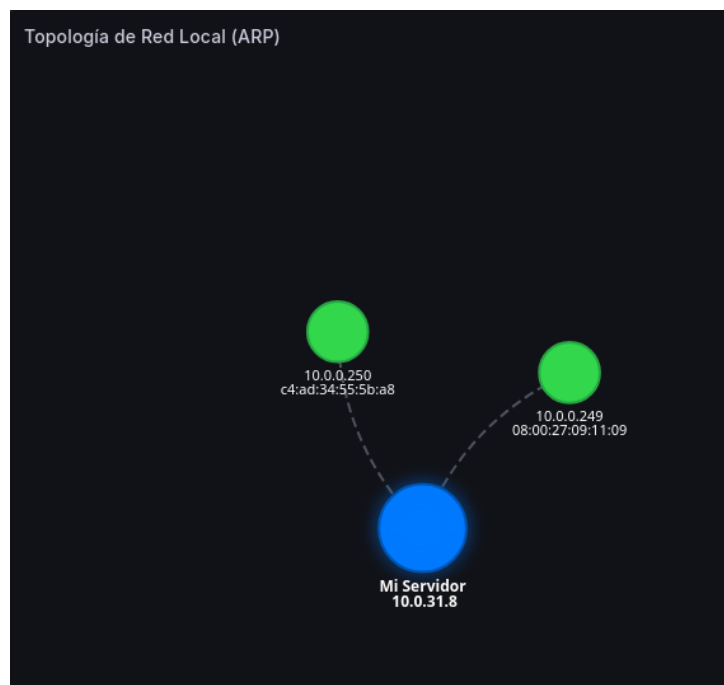


Figura 13: Topologia de red local (ARP)

Grafico que muestra la topologia de la red local (en este caso aparecen el propio servidor, grafana y mariaDB). (Uso de ECharts para mejores graficos)

4.2.6. Arbol de procesos (usando CPU)

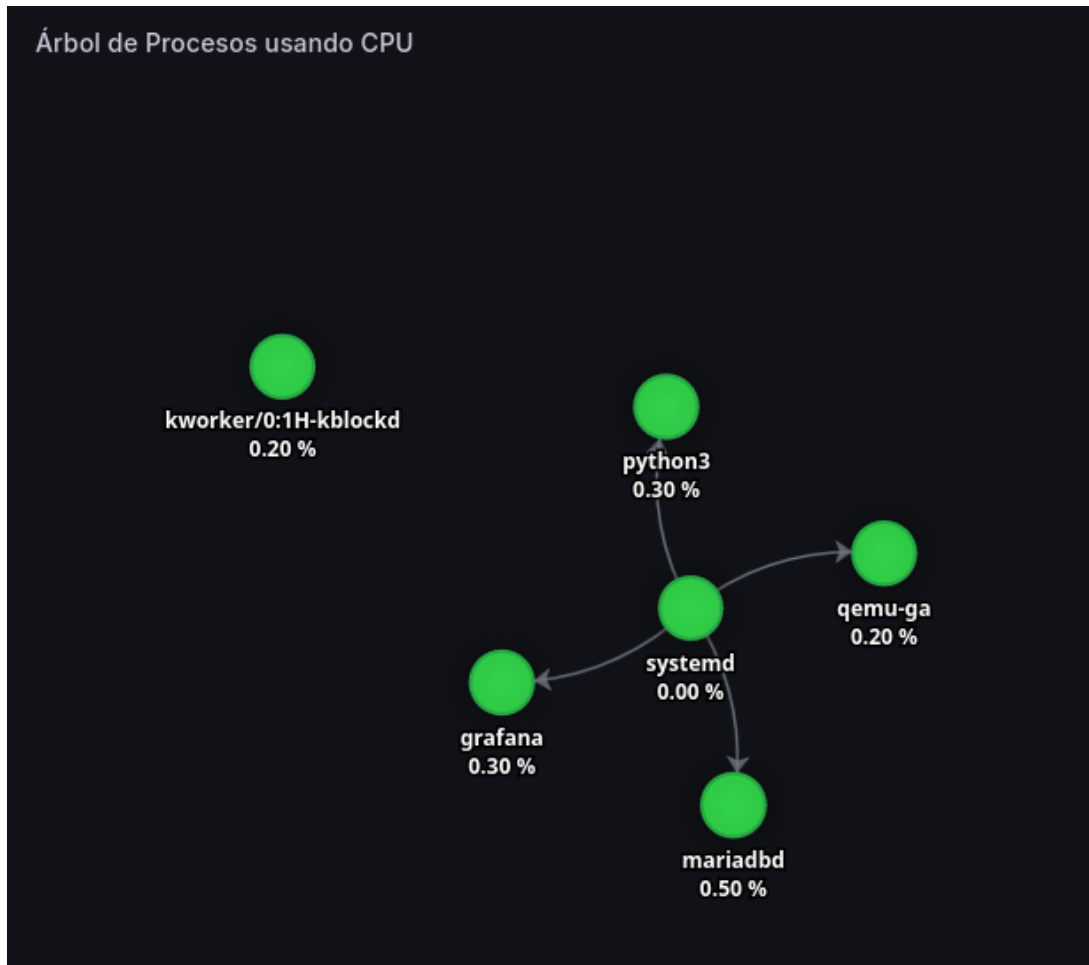


Figura 14: Arbol de procesos (usando CPU)

Grafico que muestra todos los procesos que estan haciendo uso de CPU (muchos daemons no aparecen debido a esta restriccion, si no hacen uso real de CPU no nos interesa mostrarlos). (Uso de ECharts para mejores graficos)

4.3. Visualizaciones Avanzadas (ECharts)

Para la representación de la red local y los procesos, se integró el plugin de *Business Charts* (ECharts), combinando SQL para la extracción y JavaScript para el renderizado de grafos de fuerza dirigida.

```
-- Topología de red
SELECT 'Mi Servidor' AS source, ip AS target, mac AS mac_address FROM vecinos_red;

-- Árbol de procesos (Filtrado para procesos activos)
SELECT CAST(pid AS CHAR) AS id, CAST(ppid AS CHAR) AS parent_id, nombre AS name, CAST(
  cpu_uso AS DECIMAL(10,2)) AS value FROM arbol_procesos WHERE cpu_uso > 0.0 OR pid =
  1;
```

Consultas SQL para ECharts

(El código JavaScript íntegro de ECharts utilizado para renderizar ambos grafos se encuentra configurado en las funciones del plugin en el propio dashboard de Grafana, estableciendo repulsión de nodos, colores condicionales según consumo y alertas visuales).

5. Pruebas de Estrés y Validación

Para asegurar que el dashboard refleja la realidad y el recolector funciona adecuadamente, se han elaborado scripts en Bash para someter a la máquina virtual a distintos tipos de cargas (CPU, Memoria, IO, Red y Base de Datos).

5.1. Scripts de Prueba Desarrollados

```
#!/bin/bash
# Estresa un único núcleo durante 30 segundos
taskset -c 0 stress --cpu 1 --timeout 30s
# stress --cpu 2 --timeout 30s # Para usar los dos
```

usoCPU.sh - Estrés de Procesador

```
#!/bin/bash
# Genera un pico en la cuenta de procesos activos
for i in {1..50}; do
    sleep 60 &
done
```

numProcTest.sh - Creación masiva de procesos

```
#!/bin/bash
# Crea un archivo de 500MB de golpe para disparar las alertas de almacenamiento y lectura
# /escritura
dd if=/dev/zero of=heavyFile.img bs=1M count=500
sleep 20
rm heavyFile.img
```

espacioDiscoTest.sh - Saturación de IO y Disco

```
#!/bin/bash
# Fuerza el uso de RAM al límite para provocar la activación de la partición Swap
stress --vm 2 --vm-bytes 1G --timeout 60s
```

usoRAM.sh - Saturación de RAM y Swap

```
#!/bin/bash
# Descarga un archivo grande hacia la nada (/dev/null) para generar tráfico Rx puro
echo "Generando tráfico de descarga..."
wget -O /dev/null http://speedtest.tele2.net/100MB.zip
```

usoRed.sh - Pico de Tráfico Web/Red

```
#!/bin/bash
# Usa mysqlslap para simular 50 usuarios conectados haciendo queries concurrentes (Sube las QPS)
mysqlslap --user=elliott --password=2Moronipa. --concurrency=50 --iterations=10 --auto-generate-sql
```

usoMySQL.sh - Benchmark de Base de Datos

5.2. Resultados Observados

Las reacciones del sistema monitorizadas en Grafana confirmaron el correcto funcionamiento del recolector ante cada script:

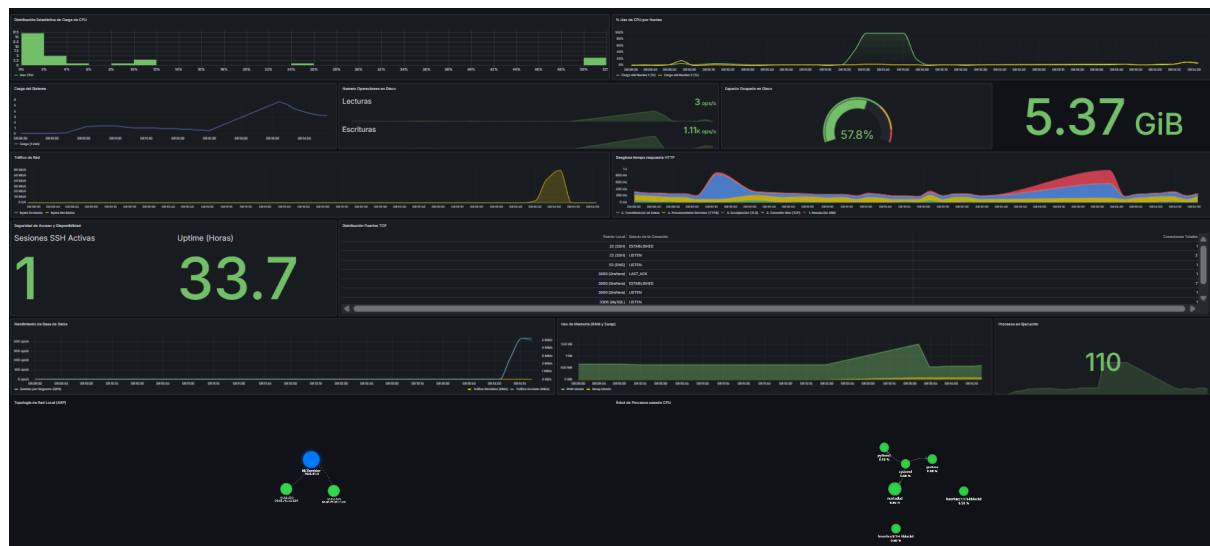


Figura 15: Picos registrados en Grafana tras ejecutar los scripts de estrés (CPU al 100 %, aumento de RAM, etc.).

- **Estrés de CPU y Load:** Al ejecutar `usoCPU.sh`, la gráfica alcanzó el 100 % y la métrica de Carga del Sistema formó una rampa ascendente.
- **Saturación de Memoria (`usoRAM.sh`):** Al exigir memoria RAM excediendo la física, la gráfica verde llega a su límite forzando al sistema a utilizar la paginación (aparición de *Swap Usado*).
- **Anomalías de Red e IO (`usoRed.sh` y `espacioDisco.sh`):** Los picos de lectura/escritura y de Bytes Recibidos se plasmaron con precisión milimétrica en sus respectivas gráficas *Time Series*.
- **Rendimiento Base de Datos (`usoMySQL.sh`):** Se registraron picos superiores a cientos de Queries Por Segundo (QPS) y la estabilización de los hilos de conexión en la gráfica de MySQL.

6. Conclusión

En conclusión, el sistema desplegado proporciona una observabilidad completa y profesional de la máquina. La incorporación de métricas avanzadas como el tráfico de red local, desglose HTTPS, monitorización de sockets TCP y análisis de MySQL elevan la práctica a un entorno aplicable al mundo laboral real. Se cumplen satisfactoriamente todos los requisitos de estabilidad, rendimiento temporal y variedad visual exigidos.